

AdoptaFácil — Documento de Especificación Funcional y Comportamiento de Interfaz (FSD) v3.5

Para el equipo de desarrollo (2 devs, React + NestJS) · 12 de septiembre de 2026

Este documento no crea requerimientos nuevos. Toma el Levantamiento V2.0 (RF01-RF28, RNF01-RNF12) y las decisiones ya consolidadas en el documento de Observaciones, y las aterriza en pantallas, esquema de base de datos exacto, jobs asíncronos y texto legal — la definición de "hecho" que el equipo necesita para construir sin ambigüedad.

⚠ Resolver antes de iniciar Ola 2 — hallazgo sobre Wompi + persona natural

Antes de la matriz de trazabilidad, esto tiene que quedar resuelto primero porque cambia el diseño del onboarding financiero de la Sección 2 y el alcance real del MVP.

Verifiqué la documentación de soporte de Wompi para la vinculación como persona natural, que es el modelo bajo el que operará AdoptaFácil mientras concluye la transición a S.A.S., y encontré dos condiciones que no estaban contempladas:

1. Retención de 30 días en el primer desembolso. La documentación de Wompi indica textualmente: *"Si te registras como persona natural, tu primer desembolso se realizará 30 días después de la primera transacción. Los siguientes desembolsos serán al siguiente día hábil."* Lo que no queda claro en la fuente, y hay que confirmar por escrito con Wompi antes de programar una sola línea del worker de dispersión, es **a qué se aplica esa ventana de 30 días**: ¿es una única vez, al activar la cuenta de AdoptaFácil como comercio? ¿O se reinicia cada vez que se dispersa por primera vez hacia un refugio nuevo? La diferencia es crítica: si se reinicia por cada refugio nuevo, significa que **cada refugio que se une a la plataforma tendría su primer mes de donaciones retenidas** en la Wompi Cuenta de AdoptaFácil antes del primer desembolso — lo cual debilita directamente el argumento de "dispersión inmediata, sin retención de saldo" sobre el que está construida toda la defensa frente al artículo 316 del Código Penal en la cláusula de mandato.

2. El modelo Agregador (el que aplica a comercios individuales/persona natural) parece limitado a Bancolombia y Nequi. La misma documentación reafirma: *"la única entidad habilitada por Wompi para recibir desembolsos es Bancolombia y aplica solo para el modelo Agregador"*, con Nequi como única alternativa para persona natural (y con 30 días mínimos de antigüedad de esa cuenta Nequi). La cobertura amplia vía Bre-B que encontramos en investigación previa puede pertenecer a un modelo comercial distinto (SPT / cuentas

negociadas), no necesariamente disponible bajo el tier de persona natural con el que arranca el MVP.

Consecuencia práctica si esto se confirma tal como se lee: durante la fase de persona natural, un refugio cuya cuenta de dispersión sea Davivienda o BBVA **no podría recibir pagos automáticos**, contradiciendo el requisito de negocio de dispersión directa a cualquier banco de uso diario del refugio. Esto no invalida la decisión de arrancar como persona natural — solo significa que el alcance de bancos soportados en el MVP puede ser más angosto de lo que se especificó, y el equipo necesita saberlo ahora, no descubrirlo con el primer refugio en Davivienda que no recibe su dispersión.

Directriz mientras se confirma con Wompi: el campo `bank_or_wallet` de `OrganizationPayoutAccount` (Sección 3) se implementa como un catálogo configurable, no como una lista fija de bancos soportados — de modo que ampliar la cobertura el día que se confirme Bre-B o se complete la transición a S.A.S. sea un cambio de configuración, no una migración de esquema. En el onboarding financiero (Sección 2.A), si el banco seleccionado no está en la lista confirmada como soportada, el sistema debe decirlo explícitamente antes de que el refugio complete el formulario, no después de que la primera dispersión falle.

SECCIÓN 1 — MAPEO DE TRAZABILIDAD (V2.0 → V3.5)

RF (V2.0)	Requerimiento original	Definición de "Hecho" en v3.5
RF01, RF02, RF03	Registro de organización; máquina de estados de formalización; gestión documental con observación/subsanación	Estado y porcentaje calculados exclusivamente desde documentos <code>APROBADO</code> (nunca escritos a mano); ciclo de 5 estados (<code>PENDIENTE</code> , <code>EN_REVISION</code> , <code>OBSERVADO</code> , <code>APROBADO</code> , <code>RECHAZADO</code>); arranque real en 0%; onboarding financiero desacoplado de la formalización tributaria, activado por evento (no en el registro)

RF (V2.0)	Requerimiento original	Definición de "Hecho" en v3.5
RF04, RF05, RF06	Dos tipos de cuenta; autenticación con verificación de correo; roles internos y matriz de permisos	Auth0 con Google Social Connection; bloqueo de toda acción de negocio hasta <code>email_verified = true</code> ; Modo Persona y Modo Organización como contextos de navegación que nunca comparten vista de datos
RF07, RF08, RF09	Expediente de animal; expediente clínico; recordatorios de vacunas	<code>Animal</code> con calendario de disponibilidad (estados <code>EN_SALIDA_TEMPORAL</code> / <code>EN_HOGAR_DE_PASO</code>); <code>AnimalBehaviorDisclosure</code> como paso bloqueante previo a cualquier <code>Placement</code> distinto de adopción, fundado en el artículo 2353 del Código Civil
RF10, RF11, RF12	Solicitud de adopción; contrato con firma electrónica; seguimiento post-adopción	Generalización de <code>AdoptionRequest</code> a <code>Placement</code> (<code>ADOPCION</code> / <code>SALIDA_TEMPORAL</code> / <code>HOGAR_DE_PASO</code>); firma electrónica simple server-side (Ley 527/1999, Decreto 2364/2012) con hash SHA-256 e inmutabilidad en bucket con Object Lock; <code>PlacementReturn</code> para el cierre con evidencia
RF13, RF14, RF26, RF28	Donaciones con desglose bruto/comisión/neto; certificado de donación; motor de pagos con dispersión; KYC/SARLAFT básico	Wompi Checkout (recaudo) + Wompi Payouts (dispersión) bajo Contrato de Mandato; <code>DonationLedger</code> de doble entrada con traslado explícito del GMF; certificado con compuerta <code>rte_vigente</code> y

RF (V2.0)	Requerimiento original	Definición de "Hecho" en v3.5
		firmante único calculado; validación cruzada de titularidad de cuenta como control KYC/SARLAFT de facto

SECCIÓN 2 — COMPORTAMIENTO PASO A PASO POR VISTAS

A. Flujo de Formalización y Dashboard del Refugio (RF01-RF03)

A.1 Estado inicial — Informal, 0% real

Al entrar por primera vez, la tarjeta principal del dashboard muestra:

Formalización

●

○

○

○

Informal · 0% completado

"Aún no has cargado ningún documento.
Empecemos por lo esencial."

[Comenzar formalización →]

El punto relleno (●) marca el paso actual, no un paso "cumplido" — a diferencia del estado actual en producción, ningún punto se pinta en verde ni lleva ícono de check en **Informal**. El 0% se calcula server-side en cada carga de página; no existe ningún valor por defecto distinto de 0 en el frontend.

A.2 CTA dinámico prioritario

Componente fijo en la parte superior del dashboard, presente en todos los estados salvo **ESAL_CON_RTE** completo sin obligaciones pendientes:

 Siguiendo paso

Paso 1 de 5 — Sube la cédula del representante legal

La necesitamos para verificar quién representa legalmente a tu organización.

[Subir documento]

El texto "Paso X de 5" y el título del paso se generan desde `ProcessTemplate.steps[current_step].title` — nunca hardcodeado en el componente. El botón deshabilita su estado de carga (`disabled` + spinner) mientras se sube el archivo, y solo se rehabilita tras confirmación del backend, no al soltar el archivo en el input.

A.3 Ciclo documental en pantalla

Cada requisito se renderiza como una fila con badge de estado:

Estado	Badge (color)	Microcopia	Acción disponible
PENDIENTE	Gris	"Pendiente de cargar"	[Subir documento]
EN_REVISION	Azul	"En revisión — normalmente respondemos en 2 días hábiles"	Ninguna (solo lectura)
OBSERVADO	Amarillo	"Necesita corrección: {motivo}" (motivo siempre visible, nunca oculto tras un clic)	[Subsanar]
APROBADO	Verde	"Verificado"	[Ver documento]
RECHAZADO	Rojo	"Rechazado: {motivo} — este documento no es válido para este requisito"	[Cargar documento nuevo]

Comportamiento del estado OBSERVADO: el motivo se muestra siempre en el cuerpo de la fila (no en un tooltip ni en un modal que haya que abrir), con un botón [Subsanar] que abre el mismo input de carga. Al subir la corrección, un Toast confirma *"Corrección enviada. La revisaremos de nuevo pronto"* y la fila pasa visualmente a **EN_REVISION** de inmediato

(optimista en el cliente, confirmado por el backend en segundo plano) — el usuario no debe quedarse mirando el badge amarillo después de haber corregido.

Distinción visual entre OBSERVADO y RECHAZADO — esto es intencional y no debe diluirse en diseño: amarillo comunica "corregible, casi listo"; rojo comunica "empieza de nuevo con este documento". Si ambos usan el mismo tono de badge, se pierde la señal que hace que el usuario sepa si debe editar o reemplazar por completo.

A.4 Onboarding financiero desacoplado (RF26, RF28)

Disparador: el modal se levanta exclusivamente cuando el usuario hace clic en una de estas tres acciones, nunca antes:

- "Publicar campaña" (primera vez)
- "Activar apadrinamientos" (primera vez)
- "Activar botón de donación pública" en el perfil de la organización

Conecta dónde quieres recibir las donaciones	
Ya verificamos tu identidad. Solo falta decirnos a qué cuenta transferimos.	
Entidad financiera	
[Selector: Bancolombia · Nequi · Daviplata · Otro banco (llave Bre-B)]	
Tipo de cuenta	
[Ahorros ○ Corriente ○ Billetera ○]	
Número de cuenta o llave Bre-B	
[]	
Titular	[precargado, solo lectura]
{nombre del representante legal / razón social}	
Documento	[precargado, solo lectura]
{NIT o cédula ya verificados}	
[Cancelar]	[Vincular cuenta]

Si el banco seleccionado no está en el catálogo confirmado como soportado (ver hallazgo al inicio de este documento), el selector lo marca con un ícono de advertencia inline y un texto

breve: "Este banco puede tener demoras adicionales en los primeros pagos — te avisaremos si aplica" — no se bloquea la selección, pero tampoco se oculta la limitación.

Precarga: `titular` y `documento` vienen de `Organization.legal_name/nit` si el RUT ya está **APROBADO**, o del nombre/cédula del representante legal si solo ese documento está aprobado. Estos dos campos se muestran como solo lectura — si el usuario necesita cambiarlos, el flujo lo redirige a corregir el documento de formalización correspondiente, no a editar el campo aquí (evita que la cuenta de pago quede con un titular distinto al verificado).

Mensaje de espera en cola de validación manual (cuando `validar_cuenta_destino` no auto-aprueba):

 Estamos verificando tu cuenta

El titular que registraste no coincide exactamente con tus documentos aprobados. Nuestro equipo lo revisará en menos de 24 horas — te escribimos apenas esté listo.

Mientras tanto, puedes seguir usando el resto de la plataforma con normalidad.

El botón de donación pública permanece oculto (no deshabilitado con explicación — directamente ausente del perfil público) mientras el estado sea distinto de **VERIFICADA**.

B. Flujo de Donación desde Redes Sociales / Campañas (RF13, RF26)

B.1 Aterrizaje (deep link desde TikTok/Instagram)

Página pública, servida con SSR (ver hallazgo de la Sección 1.1 del documento de Observaciones — esta ruta es exactamente la que debe indexar bien): foto de portada de la campaña, nombre, meta y avance en una barra de progreso, y un único CTA primario.

 Campaña: Esterilización de invierno — Hachikos

Meta: \$3.000.000 Recaudado: \$1.850.000

62%

[Donar ahora]

Sin distracciones adicionales en esta pantalla — el objetivo es que alguien que llega desde un video de 15 segundos entienda la campaña y done en menos de tres toques.

B.2 Checkout transparente (modal nativo AdoptaFácil, antes de ir a Wompi)

Este modal es intencionalmente de AdoptaFácil, no de Wompi — el desglose se muestra ANTES de invocar el checkout de la pasarela, para que la transparencia no dependa de que el usuario lea la pantalla de un tercero.

¿Cuánto quieres donar?	
[\$50.000_____]	
Tu donación	\$50.000
Costos bancarios y GMF	-\$1.400
Comisión AdoptaFácil (4%+IVA)	-\$2.380
Recibe Hachikos	\$46.220
☒ Entiendo que AdoptaFácil actúa como mandatario para recaudar y transferir esta donación a Hachikos	
[Continuar a pago seguro →]	

El desglose se recalcula en tiempo real con cada dígito que el donante escribe (debounce de ~300ms, no en cada keystroke exacto para no saturar), usando la misma fórmula de [DonationLedger](#) de la Sección 3 — nunca una aproximación redondeada distinta de la que terminará en el registro real. El checkbox de mandato **no está premarcado**; es obligatorio para habilitar "Continuar a pago seguro", y su aceptación queda registrada en [AuditLog](#) con el hash de la versión vigente del numeral 2 de los Términos y Condiciones.

B.3 Invocación de Wompi Web Checkout

Al confirmar, se abre el checkout de Wompi (modal embebido o redirección, según el SDK) con los métodos habilitados: Nequi, PSE, tarjeta de crédito/débito nacional e internacional. AdoptaFácil no recolecta ni almacena ningún dato de tarjeta — todo el formulario de pago vive dentro del iframe/redirección de Wompi (requisito de PCI DSS delegado a la pasarela, ya establecido en RNF02).

B.4 Pantalla de agradecimiento y generación de recibo

La confirmación es **asíncrona**: Wompi puede tardar unos segundos en confirmar vía webhook. La pantalla de agradecimiento se muestra con un estado transitorio:

✓ ¡Gracias por tu donación!

Estamos confirmando el pago — puede tardar unos segundos. Te enviaremos el recibo a tu correo apenas esté listo.

[Ver el progreso de la campaña]

Al recibirse el webhook de confirmación (no antes), se dispara el correo transaccional con el recibo y se actualiza la barra de progreso de la campaña en tiempo real para cualquiera que esté viendo la página en ese momento (vía polling corto o websocket, según lo que ya tenga el stack — no es necesario introducir infraestructura nueva solo para esto).

C. Flujo de Ubicación: Adopción vs. Salida de Socialización (RF10-RF12 + Placement Engine)

C.1 Página de detalle del animal — el selector que define todo lo demás

 Firulais — 2 años, mestizo

[Adoptar para siempre] [Salida de socialización]

Dos botones de peso visual similar — no se sugiere uno sobre el otro con tamaño o color, porque ambos son caminos legítimos y el refugio quiere que la gente considere el que de verdad puede sostener. El copy bajo "Salida de socialización" aclara en una línea: *"Paseos y convivencia de unos días, sin compromiso de adopción."*

C.2 Solicitud de salida de socialización

Salida de socialización con Firulais	
¿Cuándo?	
Desde [17 ^{July} 15 sep 2026]	Hasta [17 ^{July} 17 sep 2026]
Cuéntanos un poco de ti (mín. 50 caracteres)	

[Enviar solicitud]

Validación de bloqueo en tiempo real: al seleccionar el rango de fechas, el backend valida inmediatamente (no solo al enviar el formulario) si el animal ya tiene un **Placement ACTIVA** o **APROBADA** que se solape. Si hay solape, el selector de fechas muestra las fechas ocupadas deshabilitadas en el propio calendario (no un error genérico después de enviar):

Hasta [17] Calendario — 16 y 17 sep aparecen
bloqueados con tooltip "Firulais ya
tiene una salida programada"]

C.3 Paso bloqueante del refugio — declaración de comportamiento

Antes de que el refugio pueda aprobar y generar el contrato, se le presenta un modal que **no se puede omitir ni posponer**:

Antes de continuar: cuéntanos cómo es
Firulais

Esta información queda registrada como
parte del contrato y protege tanto al
refugio como a quien lo recibe.

¿Es reactivo con otros animales?

☐ No ☐ Sí, con: [_____]

¿Tiene antecedentes de mordida?

☐ No ☐ Sí, detalle: [_____]

¿Es compatible con niños?

☐ Sí ☐ Con supervisión ☐ No recomendado

Condiciones médicas relevantes para la salida
[_____]

Firma electrónica del refugio
[Nombre completo del firmante] [Firmar y continuar]

El botón "Aprobar solicitud" en la vista principal del refugio permanece **deshabilitado** hasta que este modal se complete y se firme — no es un paso posterior opcional, es la puerta de entrada a la aprobación misma. Al firmar, el mismo mecanismo de hash server-side y sellado de la

Sección 1.2 del documento de Observaciones se ejecuta sobre este documento, no solo sobre el contrato final.

C.4 Firma del comodatario

Tras la aprobación, el solicitante ve el contrato completo antes de firmar — no un resumen:

Contrato de comodato precario	
[texto completo del contrato, incluida la	
declaración de comportamiento del animal	
ya incorporada como anexo firmado]	
Entiendo que:	
☐ Respondo por el cuidado de Firulais con	
el máximo nivel de diligencia (culpa	
levísima, Art. 2203 C.C.)	
☐ El refugio puede pedir su devolución en	
cualquier momento (Art. 2219 C.C.)	
☐ Debo restituirlo el 17 sep 2026 o antes	
[Nombre completo] [Firmar]	

Los tres checkboxes son individuales y todos obligatorios — no un único "Acepto los términos" genérico, porque cada uno corresponde a una obligación civil distinta que conviene que la persona reconozca de forma explícita, no agrupada.

C.5 Paso de retorno

Vista exclusiva del refugio, disponible desde el **Placement** activo:

Registrar el regreso de Firulais	
Fotos del estado actual (mínimo 2)	
 Agregar fotos	
¿Cómo regresó?	
[]	
¿Hubo algún incidente?	
☐ No ☐ Sí — describir: []	
[Cerrar salida temporal]	

El botón "Cerrar salida temporal" permanece deshabilitado hasta que existan al menos 2 fotos cargadas. Al guardar: se crea el `PlacementReturn`, `Placement.status = CERRADA`, y `Animal.status` vuelve automáticamente a `DISPONIBLE` — sin paso manual adicional para reactivar el animal en el catálogo público. Si se marcó "Sí" en incidentes, el registro se enlaza automáticamente al expediente clínico del animal para que quede en su historial, no solo en el `Placement` cerrado.

D. Consola del Administrador de Plataforma

D.1 Cola de revisión documental

Layout de dos columnas, sin scroll independiente desalineado entre ambas:

[Visor de PDF o imagen del documento]	Documento: RUT
	Organización: Hachikos
	Subido: hace 4 horas
	☐ Nombre coincide con RUT
	☐ NIT es legible
	☐ Documento vigente
	Motivo de observación
	(obligatorio si se observa o rechaza)
	[Aprobar] [Observar]
[Rechazar]	

El campo "Motivo de observación" solo se vuelve obligatorio (borde rojo + bloqueo del botón correspondiente) cuando se presiona "Observar" o "Rechazar" — presionar "Aprobar" no lo requiere. El checklist de la derecha es orientativo para el admin, no bloquea la decisión — es una ayuda visual, no una validación de negocio.

Medición de SLA: cada fila de la cola muestra hace cuánto está esperando (`hace 4 horas`, `hace 2 días`), con un indicador de color que escala a amarillo después de 24 horas y a rojo después de 48 — visible para el propio equipo de AdoptaFácil, no para el refugio.

D.2 Cola de cuentas financieras

⚠ Titular no coincide	
Organización: Refugio Patitas	
Cuenta registrada a nombre de:	
María Fernanda Ruiz (C.C. 1.020.xxx.xxx)	
Documentos aprobados de la organización:	
Representante legal: Carlos Andrés Ruiz	
(C.C. 79.xxx.xxx)	
[Aprobar con excepción justificada]	
[Rechazar cuenta con notificación]	

"Aprobar con excepción justificada" exige un campo de texto obligatorio (ej. "cónyuge del representante legal, confirmado por WhatsApp") que queda en **AuditLog** junto con la decisión — una excepción nunca se aprueba con un solo clic sin dejar constancia de por qué. "Rechazar cuenta con notificación" dispara automáticamente un correo al refugio explicando el motivo y ofreciendo registrar una cuenta distinta.

SECCIÓN 3 — ARQUITECTURA DE DATOS, LEDGER Y MOTOR ASÍNCRONO

3.1 Esquema relacional extendido (Prisma / PostgreSQL)

```
enum PlacementType {  
  ADOPCION  
  SALIDA_TEMPORAL  
  HOGAR_DE_PASO  
}
```

```
enum PlacementStatus {  
  SOLICITADA  
  EN_EVALUACION  
  APROBADA  
  ACTIVA  
  CERRADA  
  CANCELADA
```

```
VENCIDA_SIN_RETORNO
}
```

```
model Placement {
  id          String      @id @default(uuid())
  placementType PlacementType
  animalId    String
  animal      Animal      @relation(fields: [animalId], references: [id])
  requesterUserId String
  requester   User        @relation(fields: [requesterUserId], references: [id])
  organizationId String
  organization Organization @relation(fields: [organizationId], references: [id])
  startDate   DateTime
  endDate     DateTime?    // NULL únicamente si placementType = ADOPCION
  status       PlacementStatus @default(SOLICITADA)
  contractId  String?      @unique
  contract     AdoptionContract? @relation(fields: [contractId], references: [id])
  behaviorDisclosureId String? // obligatorio si placementType != ADOPCION
  behaviorDisclosure AnimalBehaviorDisclosure? @relation(fields: [behaviorDisclosureId],
references: [id])
  createdAt    DateTime    @default(now())
  updatedAt    DateTime    @updatedAt

  @@index([organizationId])
  @@index([animalId, status])
  @@index([status, endDate]) // usado por el job de vencimiento de retorno
  @@check(constraint: "end_date_required_unless_adopcion",
    sql: "(placement_type = 'ADOPCION' AND end_date IS NULL) OR (placement_type !=
'ADOPCION' AND end_date IS NOT NULL)")
}
```

```
model AnimalBehaviorDisclosure {
  id          String      @id @default(uuid())
  animalId    String
  animal      Animal      @relation(fields: [animalId], references: [id])
  organizationId String
  declaredByUserId String
  reactivityNotes String?
  biteHistory  Boolean    @default(false)
  biteHistoryDetail String?
  childrenCompatibility String // enum a nivel de aplicación: SI | CON_SUPERVISION |
NO_RECOMENDADO
  medicalConditionsRelevant String?
```

```

    declaredAt          DateTime @default(now())
    signatureEventId    String  @unique
    signatureEvent      SignatureEvent @relation(fields: [signatureEventId],
references: [id])
    placements          Placement[]

    @@index([animalId])
}

model PlacementReturn {
    id          String  @id @default(uuid())
    placementId String  @unique
    placement   Placement @relation(fields: [placementId], references: [id])
    returnedAt  DateTime @default(now())
    physicalConditionNotes String
    photoEvidence String[] // mínimo 2, validado a nivel de aplicación
    verifiedByUserId String
    incidentReported Boolean @default(false)
    incidentDetail String?
    linkedClinicalEventId String? // FK a ClinicalEvent si incidentReported = true

    @@check(constraint: "min_two_photos", sql: "array_length(photo_evidence, 1) >= 2")
}

enum PayoutStatus {
    PENDING
    PROCESSING
    SETTLED
    REJECTED
}

model DonationLedger {
    id          String  @id @default(uuid())
    donationId  String  @unique
    donation    Donation @relation(fields: [donationId], references: [id])
    organizationId String
    organization Organization @relation(fields: [organizationId], references: [id])

    grossAmount      Decimal @db.Decimal(14, 2)
    gatewayFee        Decimal @db.Decimal(14, 2)
    gmfTransferFee    Decimal @db.Decimal(14, 2)
    payoutTransferFee Decimal @default(0) @db.Decimal(14, 2)
    platformFeeNet    Decimal @db.Decimal(14, 2) // 4% sobre grossAmount

```

```

platformFeeIva                Decimal @db.Decimal(14, 2) // 19% sobre platformFeeNet
netPayableRefugio             Decimal @db.Decimal(14, 2)

payoutStatus                   PayoutStatus @default(PENDING)
wompiTransactionId             String? @unique
payoutBatchId                  String?
payoutReferenceWompi           String? @unique

createdAt                      DateTime @default(now())
settledAt                      DateTime?

@@index([organizationId, payoutStatus])
@@index([payoutBatchId])
@@check(constraint: "net_payable_matches_formula",
        sql: "net_payable_refugio = gross_amount - gateway_fee - gmf_transfer_fee -
payout_transfer_fee - platform_fee_net - platform_fee_iva")
}

enum PayoutAccountStatus {
    PENDIENTE
    VERIFICADA
    RECHAZADA
}

model OrganizationPayoutAccount {
    id                String @id @default(uuid())
    organizationId    String @unique // una cuenta activa a la vez por organización
    organization      Organization @relation(fields: [organizationId], references: [id])
    accountHolderName String
    accountHolderDocument String
    bankOrWallet      String // catálogo configurable, no enum fijo — ver hallazgo
    Wompi
    accountNumberOrKey String @db.Text // cifrado a nivel de aplicación antes de
persistir
    accountType        String // AHORROS | CORRIENTE | BILLETERA
    verificationStatus  PayoutAccountStatus @default(PENDIENTE)
    verifiedAt          DateTime?
    createdAt           DateTime @default(now())

    @@index([organizationId])
}

```

Notas de implementación que no son obvias desde el DDL:

- El `@@check` de `Placement` es la barrera de base de datos contra el error más probable de este módulo: alguien seteando `endDate` en una adopción o dejándolo nulo en una salida temporal. No confiar solo en la validación del DTO de NestJS — la constraint debe existir también en Postgres.
- El `@@check` de `DonationLedger` (`net_payable_matches_formula`) es la garantía de que ningún bug de cálculo en el backend puede persistir un ledger cuya suma no cuadre — si la aritmética falla, el `INSERT` falla, no se guarda un registro contable incorrecto.
- `accountNumberOrKey` se declara `@db.Text` para admitir tanto números de cuenta como llaves Bre-B (que pueden ser un correo o un celular, no un número fijo de dígitos) — cifrado en la capa de aplicación antes del `INSERT`, nunca en texto plano en la base de datos.
- `OrganizationPayoutAccount.organizationId` es `@unique`: una organización tiene una sola cuenta de dispersión activa a la vez. Un cambio de cuenta es un `UPDATE` auditado (ver Sección D.2), no una nueva fila — así el historial de auditoría queda en `AuditLog`, no disperso en múltiples registros de la misma tabla.

3.2 Worker de Dispersión Nocturna (Payouts)

WORKER: payout-dispatcher

TRIGGER: automático, al confirmarse cada `DonationLedger` vía webhook de recaudo (no es estrictamente "nocturno" — se dispara inmediato por donación, con reintentos escalonados si Wompi no responde; el batch nocturno es solo para consolidar múltiples donaciones pendientes de un mismo refugio en un único lote cuando el volumen lo justifique)

PASO 1 — Agrupación de saldos por organización

```
SELECT * FROM DonationLedger
WHERE payoutStatus = 'PENDING'
GROUP BY organizationId
```

PASO 2 — Validación de cuenta verificada (compuerta bloqueante)

FOR cada organización con saldo pendiente:

IF `OrganizationPayoutAccount.verificationStatus` != 'VERIFICADA':

- NO se dispersa. El saldo permanece en `PENDING`.
- Se genera una `Obligation` tipo `PAYOUT_BLOCKED_NO_ACCOUNT` (entra al motor de notificaciones existente, audiencia `ORG_ADMINS`)
- Este caso NO es un error del job — es un estado esperado mientras el refugio completa su onboarding financiero

PASO 3 — Invocación de Wompi Payouts

POST `/payouts` (uno a uno) o `/payouts/file` (por lote, formato según banco destino — ver catálogo configurable de `bankOrWallet`)

payload incluye: monto = netPayableRefugio, cuenta destino, referencia
idempotency_key = donationLedgerId -- evita doble dispersión si el job
se reintenta tras timeout

PASO 4 — Conciliación por webhook

ON webhook 'payout.approved':

DonationLedger.payoutStatus = 'SETTLED'

DonationLedger.settledAt = now()

ON webhook 'payout.declined':

DonationLedger.payoutStatus = 'REJECTED'

→ Obligation tipo PAYOUT_FAILED, audiencia ORG_ADMINS + PLATFORM_ADMIN

→ el monto NO se reintenta automáticamente hacia la misma cuenta;
requiere revisión manual antes de reintentar (evita reintentos
ciegos hacia una cuenta que Wompi ya rechazó)

MANEJO DE FALLOS

- Timeout de red al llamar la API de Wompi: reintento con backoff exponencial (3 intentos: 1min, 5min, 20min), usando el mismo idempotency_key en cada intento
- Tras 3 fallos: PayoutStatus permanece PENDING, se genera alerta interna (no al refugio todavía) para revisión de infraestructura — distinto del caso de RECHAZADO, que sí es una respuesta explícita de Wompi y sí se comunica al refugio

3.3 Worker de Notificaciones e Idempotencia

WORKER: obligation-notifier

TRIGGER: cron nocturno (una vez al día es suficiente para la mayoría de offsets; los de vencimiento crítico como PAYOUT_FAILED se evalúan además en cada ciclo corto del worker de dispersión)

PASO 1 — Evaluar Obligation.dueAt contra hoy + cada offset de NotificationRule

```
matches = Obligation.filter(o =>  
  NotificationRule[o.type].offsets.any(offset =>  
    today == o.dueAt + offset  
  )  
)
```

PASO 2 — Por cada match, ENCOLAR (no enviar directamente)

FOR match IN matches:

```
  ENQUEUE notification-send-job({  
    ruleId, obligationId: match.id, offset, channel  
  })
```

PASO 3 — Clave de idempotencia — por qué previene duplicados ante un reinicio de pod

Antes de encolar, se hace un UPSERT contra una tabla de control:

```
CREATE TABLE sent_notifications (  
  rule_id    TEXT,  
  obligation_id TEXT,  
  offset_value TEXT,  
  channel    TEXT,  
  sent_at    TIMESTAMPTZ,  
  PRIMARY KEY (rule_id, obligation_id, offset_value, channel)  
);  
  
INSERT INTO sent_notifications (...) VALUES (...)  
ON CONFLICT (rule_id, obligation_id, offset_value, channel) DO NOTHING  
-- si el INSERT no afecta ninguna fila (porque ya existía la clave),  
-- el job NO encola el envío. Esto es lo que hace que sea seguro que  
-- Kubernetes mate el pod a mitad del cron y lo reinicie: el segundo  
-- pod vuelve a evaluar los mismos matches, pero el UPSERT bloquea el  
-- reenvío de lo que el primer pod ya alcanzó a marcar como enviado,  
-- incluso si el envío real del correo falló después del INSERT  
-- (ese caso de borde — INSERT exitoso pero envío fallido — se  
-- resuelve con un job de reconciliación separado que revisa  
-- sent_notifications sin confirmación de entrega del proveedor de  
-- correo y las reintenta explícitamente, no de forma automática  
-- en el próximo ciclo del cron principal)
```

SECCIÓN 4 — CLÁUSULAS ESPECÍFICAS: MANDATO, COMODATO Y COSTOS FINANCIEROS

Nota de implementación antes del texto: la cláusula de mandato de esta sección reemplaza la del documento de Observaciones únicamente en lo referente a la identificación de la parte operadora, para reflejar la operación inicial bajo persona natural. El resto de la arquitectura de mandato (ingresos recibidos para terceros, facturación solo sobre la comisión, no captación) no cambia.

Cláusula — Contrato de mandato para recaudo y dispersión (identificación bajo persona natural)

Identificación de la parte operadora. Durante la etapa inicial de operación, y hasta que se perfeccione la constitución y puesta en marcha de la sociedad por acciones simplificada a través de la cual AdoptaFácil operará en adelante, los servicios de recaudo y dispersión descritos en esta cláusula son prestados por **[Nombre completo], identificado(a) con cédula de ciudadanía No. [número], persona natural que ejerce la actividad comercial de desarrollo de sistemas informáticos y procesamiento de datos registrada ante la DIAN bajo los códigos CIU 6201 y 6311, actuando bajo la marca comercial "AdoptaFácil"** ("el Operador"). Toda referencia a "AdoptaFácil" en estos Términos, en lo relativo a la gestión de recaudo y dispersión de fondos, debe entenderse hecha al Operador así identificado, sin perjuicio de que otras funcionalidades de la plataforma puedan estar habilitadas por un vehículo societario distinto.

Objeto del mandato. Al habilitar su cuenta para recibir donaciones, campañas o apadrinamientos, la Organización confiere al Operador un mandato con representación, en los términos del artículo 2142 del Código Civil, con el único objeto de: (i) recibir, por cuenta y en nombre de la Organización, las sumas que los donantes le destinen a través de la plataforma; (ii) descontar de dichas sumas la remuneración pactada y los costos de procesamiento y transferencia bancaria correspondientes; y (iii) transferir el saldo neto a la cuenta que la Organización haya registrado y verificado.

Naturaleza de los fondos. Las sumas recibidas por el Operador a través de la pasarela de pagos no constituyen ingreso propio del Operador en ningún momento, sino recursos de titularidad de la Organización recibidos para su transferencia, conforme a la doctrina de la Dirección de Impuestos y Aduanas Nacionales sobre ingresos recibidos para terceros en contratos de mandato (entre otros, Oficio DIAN 907500 de 2021 y Oficio DIAN 27842 de 2019). El Operador transferirá dichas sumas de forma inmediata o dentro del menor plazo que la infraestructura bancaria y de la pasarela de pagos permita, sin retener saldo alguno más allá del tiempo estrictamente necesario para el procesamiento.

Transición societaria. La Organización acepta desde ahora que, una vez el Operador constituya la sociedad a través de la cual AdoptaFácil continuará operando, el mandato aquí conferido se entenderá cedido a dicha sociedad en los mismos términos, sin que sea necesaria una nueva aceptación de estos Términos para efectos de continuidad del servicio, salvo por la actualización informativa que AdoptaFácil comunicará a través de la plataforma indicando la nueva identificación del Operador. La Organización podrá, en todo caso, solicitar la terminación del mandato en cualquier momento, conforme a lo dispuesto en el numeral correspondiente de estos Términos.

Naturaleza personal de la operación durante esta etapa — declaración de transparencia. La Organización reconoce haber sido informada de que, durante esta etapa, los servicios de recaudo y dispersión son prestados por una persona natural y no por una sociedad con

patrimonio separado, y que ello no modifica las garantías de no captación, transparencia en el desglose de costos, ni inmediatez en la dispersión establecidas en estos Términos.

Cláusula — Comodato precario para salidas de socialización

(Sin cambios respecto al texto ya incorporado en la Sección 3, numeral 9, del documento de Observaciones — se reproduce aquí para que el equipo tenga el texto completo en un solo documento de referencia durante la construcción.)

Cuando una Organización entregue temporalmente un animal a un Usuario Persona para salidas de socialización, convivencia temporal o acogida en hogar de paso, dicha entrega constituye un contrato de comodato o préstamo de uso en los términos del artículo 2200 del Código Civil. La Organización conserva la propiedad del animal y entrega únicamente su tenencia por el tiempo convenido, con cargo de restitución. El comodato se pacta con carácter precario (artículo 2219 del Código Civil): la Organización puede solicitar la restitución en cualquier tiempo. El tenedor temporal responde hasta de culpa levísima (artículo 2203 del Código Civil). Previo a toda entrega temporal, la Organización debe declarar formalmente el comportamiento conocido del animal; su omisión impide la generación del contrato. Durante la tenencia temporal, el Usuario Persona responde por los daños que el animal cause a terceros conforme al artículo 2353 del Código Civil, sin perjuicio de su acción contra la Organización si el daño proviene de una calidad o vicio del animal que esta debió declarar y no declaró. AdoptaFácil no es parte del contrato de comodato y no responde por los daños ni por el incumplimiento de la obligación de restitución.

Cláusula — Traslado de costos financieros y GMF (4x1000)

El monto que el donante paga se ve afectado por los siguientes descuentos, aplicados en el orden aquí descrito: primero, la comisión de procesamiento de la pasarela de pagos; segundo, la comisión de intermediación de AdoptaFácil del cuatro por ciento (4%) más IVA, calculada sobre el monto bruto de la donación; tercero, el Gravamen a los Movimientos Financieros (4x1000) causado por la transferencia de los fondos hacia la cuenta de la Organización, junto con cualquier costo operativo adicional de dispersión bancaria que la pasarela cobre por dicho concepto. La comisión de intermediación de AdoptaFácil constituye su única contraprestación por el servicio prestado; la comisión de la pasarela y el GMF son costos reales de terceros que se trasladan a la Organización como parte del costo de operar la dispersión, y en ningún caso se descuentan de la comisión de AdoptaFácil ni la afectan. El desglose completo de estos conceptos se muestra al donante y a la Organización antes de confirmar cada transacción y queda disponible en el historial de cada una.

SECCIÓN 5 — CHECKLIST TÉCNICO DE ACEPTACIÓN BLOQUEANTE (QA DEFINITION OF DONE)

#	Requerimiento	Criterio Técnico (Given–When–Then)	Prueba Bloqueante
1	Cálculo de formalización derivado	Given cero documentos APROBADO , When se consulta el dashboard, Then el porcentaje es exactamente 0 y el estado es INFORMAL sin marca de completado	Query directa a la base de datos confirma que no existe ninguna ruta de UPDATE sobre el campo de porcentaje fuera de la función de cálculo derivado
2	Bloqueo de contrato temporal sin declaración firmada	Given un Placement de tipo SALIDA_TEMPORAL sin behaviorDisclosureId , When el refugio intenta generar el contrato, Then la operación se rechaza en backend	Llamar el endpoint de generación de contrato directamente (sin pasar por la UI) sin behaviorDisclosureId debe devolver error 4xx explícito, incluso para un rol OWNER de la organización
3	Exclusión del catálogo público durante salida temporal	Given un Placement en estado ACTIVA , When se consulta el catálogo público de animales, Then ese animal no aparece en los resultados	Test automatizado que verifica ausencia del animalId en la respuesta del endpoint público mientras exista un Placement ACTIVA asociado

#	Requerimiento	Criterio Técnico (Given–When–Then)	Prueba Bloqueante
4	Bloqueo de solapamiento de fechas	Given un animal con Placement ACTIVA o APROBADA en un rango de fechas, When se intenta crear un segundo Placement con rango solapado, Then la creación falla por validación de negocio	Intentar crear dos Placement con fechas cruzadas sobre el mismo animalId vía llamada directa a la API debe fallar con mensaje explícito, no con error 500 genérico
5	Inmutabilidad del contrato sellado	Given un AdoptionContract o contrato de comodato en estado FIRMADO , When cualquier actor (incluido SuperAdmin) intenta modificarlo, Then la operación se rechaza a nivel de base de datos	UPDATE directo contra la fila debe fallar por trigger o política de base de datos, no solo por ausencia de botón en la UI; el objeto en el bucket debe rechazar sobrescritura por política de Object Lock
6	Cálculo matemático del DonationLedger	Given una donación de monto bruto conocido, When se liquida, Then $\text{platformFeeNet} = \text{grossAmount} * 0.04$, $\text{platformFeeIva} = \text{platformFeeNet} * 0.19$, y netPayableRefugi o cumple exactamente la fórmula de la constraint	Insertar un DonationLedger con una suma que no cuadre debe fallar por la constraint de base de datos, no solo por validación de aplicación

#	Requerimiento	Criterio Técnico (Given-When-Then)	Prueba Bloqueante
		<code>net_payable_matches_formula</code>	
7	GMF trasladado, comisión de plataforma intacta	Given dos donaciones idénticas en monto bruto pero con <code>gmfTransferFee</code> distinto, When se comparan sus ledgers, Then <code>platformFeeNet</code> es idéntico en ambas y toda la diferencia recae en <code>netPayableRefugio</code>	Variar <code>gmfTransferFee</code> en un entorno de prueba y confirmar que <code>platformFeeNet</code> no cambia en ningún escenario
8	Aislamiento multi-tenant (RLS)	Given un usuario autenticado en la organización A, When intenta leer un <code>DonationLedger</code> o <code>Placement</code> de la organización B por consulta SQL directa (no por la API), Then la respuesta es vacía	Ejecutar la consulta directamente contra Postgres con el rol de aplicación (no el rol <code>BYPASSRLS</code>) y confirmar 0 filas devueltas para datos de otro tenant
9	Pre-renderizado / SSR de rutas públicas	Given la ruta pública de una campaña o del catálogo de animales, When se hace una petición HTTP sin ejecutar JavaScript, Then el HTML devuelto contiene título, meta descripción y contenido visible	<code>curl</code> sin motor de JS a la ruta pública debe mostrar el nombre de la campaña o del animal en el HTML crudo, no un contenedor vacío

#	Requerimiento	Criterio Técnico (Given–When–Then)	Prueba Bloqueante
10	Redirección 301 del hostname de Azure	Given una petición al hostname por defecto de Azure, When se resuelve, Then responde 301 hacia https://www.adop.tafacil.org	Verificar código HTTP 301 y cabecera Location en staging y producción
11	Compuerta de dispersión sin cuenta verificada	Given una organización con saldo pendiente y OrganizationPaymentAccount.verificationStatus != VERIFICADA , When corre el worker de dispersión, Then el saldo permanece PENDING y se genera la Obligation PAYOUT_BLOCKED_NEW_ACCOUNT sin intentar la llamada a Wompi	Simular el escenario en staging y confirmar que no se genera ninguna llamada saliente a la API de Payouts mientras la cuenta no esté verificada
12	Idempotencia de dispersión ante reintento	Given un timeout de red al llamar a Wompi Payouts, When el worker reintenta con el mismo idempotency_key , Then Wompi no ejecuta una segunda dispersión para el mismo donationLedgerId	Forzar dos llamadas consecutivas con el mismo idempotency_key en el ambiente de pruebas de Wompi y confirmar una sola transacción de dispersión registrada

#	Requerimiento	Criterio Técnico (Given–When–Then)	Prueba Bloqueante
13	Idempotencia del motor de notificaciones	Given una Obligation con una regla y offset que ya generaron un envío, When el cron corre nuevamente tras un reinicio simulado del pod, Then no se genera un segundo envío para la misma combinación (rule_id , obligation_id , offset , channel)	Matar el proceso del worker a mitad de un ciclo de envío en staging, reiniciarlo, y confirmar en sent_notifications que no hay filas duplicadas ni correos repetidos en el proveedor de envío
14	Cierre de Placement exige evidencia mínima	Given un Placement temporal activo, When se intenta cerrar sin al menos 2 fotos en photoEvidence , Then la operación falla por la constraint min_two_photos	Intentar insertar un PlacementReturn con 0 o 1 foto debe fallar a nivel de base de datos, no solo de formulario
15	Retorno reactiva el animal automáticamente	Given el cierre exitoso de un Placement temporal, When se registra el PlacementReturn , Then Animal.status vuelve a DISPONIBLE sin acción manual adicional	Verificar en staging que el animal reaparece en el catálogo público inmediatamente después del cierre, sin necesidad de que el admin lo reactive por separado

Fin del FSD v3.5. Documento vivo — cualquier ajuste de alcance durante la construcción debe reflejarse aquí antes de mergear, no documentarse después.